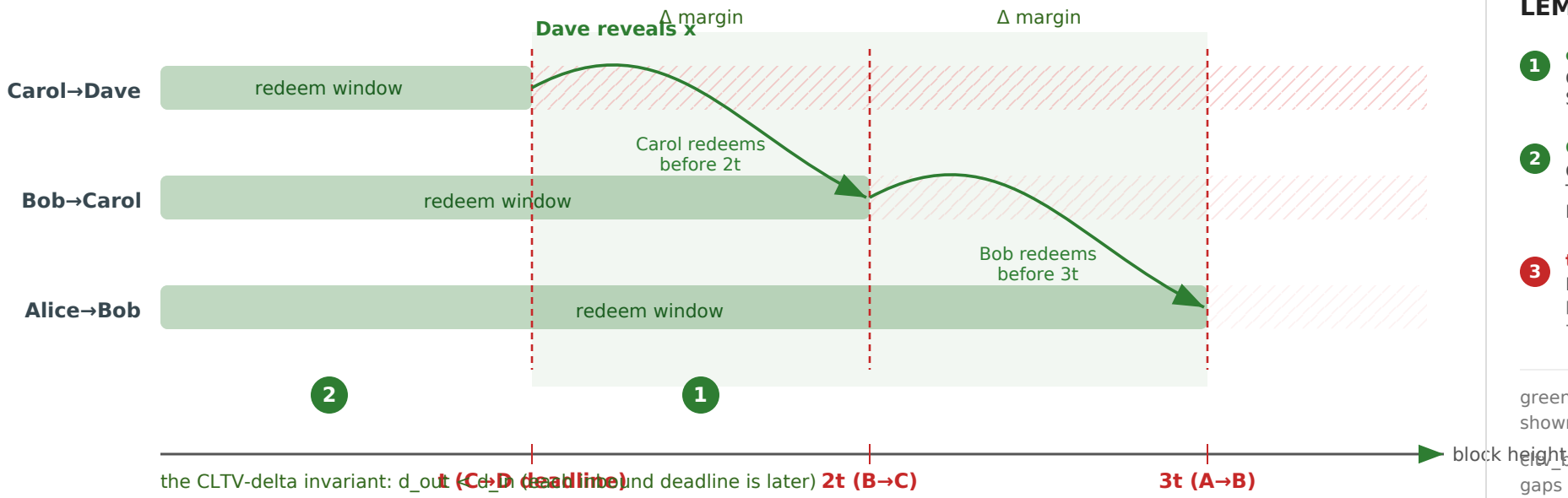


Staggered CLTV timelocks — timing safety (block height →)

covers cltv_blocks.spthy · Clock.spthy · timeout.spthy — see rail →



LEMMAS · timing theories

- 1 **cltv_blocks.spthy — windows EXIST**
CLTV_Gap_Is_Positive ✓ · Claim_Window_Nonempty ✓
Staggered_Path_Safe ✓ (derives $d_{out} < d_{in}$ from block)
- 2 **Clock.spthy — claim-in-time + lifecycle**
Claim_Window_Exists ✓ · Outcome_Exclusive ✓
Transitive_Preimage_Before_Upstream_Deadline ✓
No_HTLC_After_Close ✓ · Redeem_Reachable / Refund_R
- 3 **timeout.spthy — the race (T2b removed)**
Early_Timeout_Race ✓ (reachable without T2b)
pairs with multihop's Timeout_Race_Blocked ✓
→ together: proof that T2b is necessary, not assumed aw

green = timing safety proved · red = the race, shown reachable to justify the assumption.
cltv_blocks derives what multihop assumes; gaps adds the block-clock lifecycle safety.

- 3 **WITHOUT the liveness assumption T2b (timeout.spthy)**
adversary fires the A→B inbound timeout early → Bob's outbound is redeemed but his inbound refunds → Bob loses.
- Bob's outbound redeemed (paid out)
- inbound timed out EARLY → Bob refunds inbound = net loss
- Two-sided certificate: race reachable here (T2b removed) ↔ blocked in multihop.spthy (Timeout_Race_Blocked). So T2b is load-bearing.